

Machine Learning-Based Prediction of BLEVE Target Pressure Report

Author : Jian Heng Koo

Date : 10th May 2026

Table of Contents

1. COMPETITION RESULTS	3
2. INTRODUCTION	3
3. DATA CLEANING.....	4
3.1. MISSING VALUES AND THE 27-SENSOR STRUCTURE.....	4
3.2. DUPLICATE ROWS	4
3.3. INCOMPLETE EVENTS.....	4
3.4. INCORRECT ENTRIES AND UNIT CHECKS	4
3.5. OUTLIERS CHECKING	5
4. DATA PREPROCESSING	5
4.1. EVENT ASSIGNMENT	5
4.2. EVENT-LEVEL DENOISING	5
4.3. FEATURE ENGINEERING	6
4.4. ENCODING AND DATA TYPE CONVERSION.....	6
4.5. SCALING AND TARGET TRANSFORMATIONS.....	7
5. MODEL SELECTION.....	7
5.1. LINEAR REGRESSION MODEL.....	7
5.2. RANDOM FOREST MODEL.....	7
5.3. NEURAL NETWORK MODEL.....	7
5.4. BLENDED ENSEMBLE MODEL	7
6. HYPERPARAMETER TUNING.....	8
6.1. VALIDATION STRATEGY	8
6.2. SEARCH STRATEGY	9
6.3. TUNED HYPERPARAMETERS AND CALIBRATION	10
7. MODEL PREDICTION PERFORMANCE	10
7.1. LOCAL VALIDATION PERFORMANCE	10
7.2. KAGGLE PUBLIC TEST PERFORMANCE.....	10
8. REFLECTION	11
8.1. CHALLENGES	11
8.2. LEARNING.....	12
8.3. IMPROVEMENTS.....	12
9. REFERENCES	13

1. Competition Results

4



Your Best Entry!

Your most recent submission scored 0.13285,

Best public leaderboard MAPE: 0.13285

Public leaderboard position: 4th

Final leaderboard position: 10th out of 448 entries

2. Introduction

This project focuses on predicting the peak pressure produced in Boiling Liquid Expanding Vapour Explosion (BLEVE) blast scenarios. The dataset provides physical setup parameters and sensor readings around an obstacle wall, and the goal is to map those variables accurately to the resulting target pressure in bars.

While developing the models, I used generative AI assistance to help organise the Jupyter notebook and debug specific functions. I still reviewed, audited, and executed all final code myself. The analytical strategy and methodological decisions were my own.

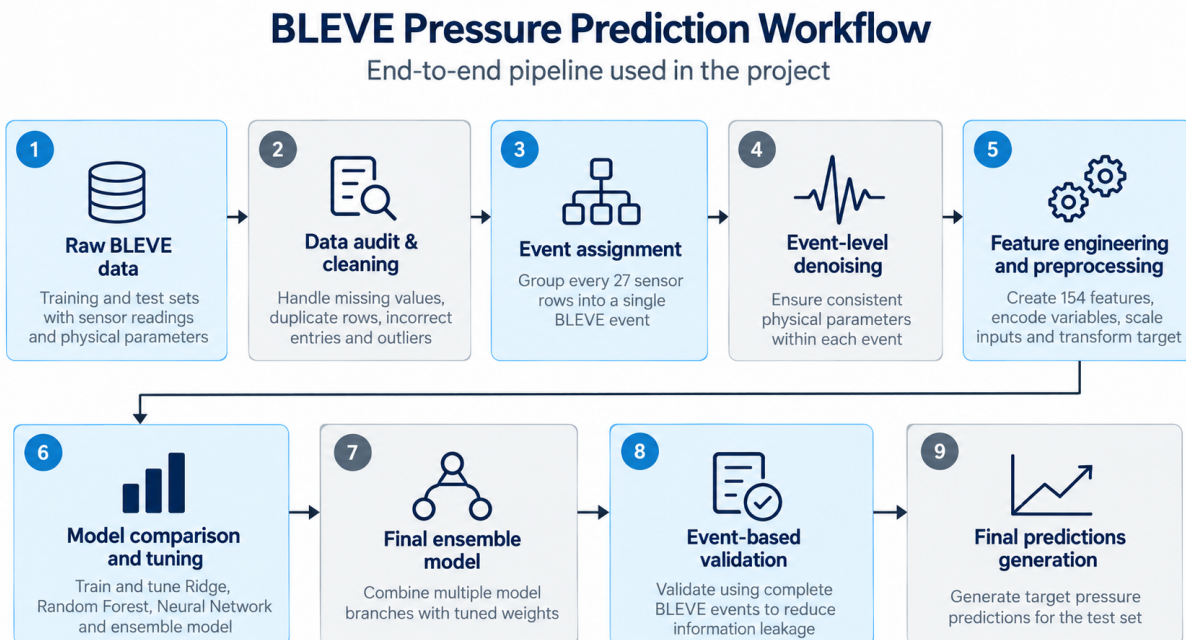


Figure 1- End-to-end BLEVE pressure prediction workflow from raw data preparation to final test-set predictions.

3. Data Cleaning

An initial audit of the 10,050 training rows and 3,203 testing rows showed several data quality issues that needed to be cleaned before modelling could begin.

3.1. Missing Values and the 27-sensor Structure

The training data contained 302 missing values across 112 rows. Rows without the actual target pressure were removed so the model would only learn from reliable labelled data. Filling a missing target, even with a median value, would create artificial labels and teach the model false patterns.

An important early finding was that the dataset was not just a continuous list of unrelated sensor rows. It followed a strict 27-sensor pattern for each BLEVE event. By decomposing the row ID, I could identify both the specific explosion and the sensor position within that event.

- $\text{Event_ID} = \text{ID} // 27$
- $\text{Event_Pos} = \text{ID} \% 27$ (ranging from 0 to 26)

This structure became the basis of the cleaning process. Physical setup variables such as tank dimensions and initial temperature should be identical for every sensor recording the same explosion, so missing values in these shared parameters could be patched using the event-wise median.

3.2. Duplicate Rows

The training set contained 50 exact duplicate rows. I removed them before model fitting because repeated rows can give the same sensor reading too much influence and make validation less reliable.

3.3. Incomplete Events

To handle this consistently, I used a general completeness check for training events. After missing-target and duplicate-row removal, any training event that no longer contained the full 27-sensor layout was treated as incomplete. This also included the boundary case where one BLEVE event was split across the provided testing and training files.

The condition was simple: a training event was kept only if it had 27 rows, 27 unique Event_Pos values, a minimum Event_Pos of 0, and a maximum Event_Pos of 26. Applying this same rule to all training events removed 11 incomplete training events and 270 rows, leaving 9,720 cleaned training rows across 360 complete training events. The test set was kept intact because Kaggle requires a prediction for every test row.

3.4. Incorrect Entries and Unit Checks

The Status feature contained inconsistent spellings, including subcool, subcooled, superheat, and saperheated. I mapped these into two numeric classes using close string matching: subcooled = 0 and superheated = 1.

Original meaning	Encoded value
Subcooled	0
Superheated	1

I also applied a rule-based correction to Tank Failure Pressure values that appeared to be stored at the wrong scale. Values greater than five times the median were divided by 100. This was a conservative correction for clearly inconsistent pressure-scale values.

The boiling and critical temperature columns were treated as Celsius because some values are negative, which would not be physically possible in Kelvin. I converted these columns internally to Kelvin by adding 273.15 before calculating thermodynamic ratios.

3.5. Outliers Checking

I used interquartile range (IQR) checks to identify unusually large or small values. The largest outlier counts appeared in sensor position, vapour height, and tank failure pressure features. I did not simply remove these values because BLEVE pressure data can naturally contain extreme but meaningful cases. Instead, robust scaling and target transformations were used later to reduce the influence of extreme values.

4. Data Preprocessing

The preprocessing stage turned the row-based data into an event-aware modelling dataset. This included event grouping, event-level denoising, type conversion, feature engineering, encoding, scaling, and target transformation.

4.1. Event Assignment

Each BLEVE scenario has up to 27 sensor rows. The notebook creates Event_ID with $ID // 27$ and Event_Pos with $ID \% 27$. Event_ID identifies the BLEVE scenario a row belongs to, while Event_Pos gives the zero-based position inside the 27-row event. The actual Sensor ID feature remains in its original 1 to 27 range.

Event assignment matters for two reasons. First, it allows shared setup variables to be cleaned within each event. Second, it allows validation to hold out complete events rather than mixing sensors from the same event into both training and validation.

4.2. Event-Level Denoising

After event assignment, I replaced shared numeric setup variables with the median value within the same Event_ID. This reduces row-level noise in variables that should be constant across all sensors in the same BLEVE scenario. Sensor-specific variables such as Sensor ID, sensor side, sensor x/y/z position, and target pressure were not smoothed because they are expected to vary by sensor.

4.3. Feature Engineering

Feature group	Examples	Purpose
Geometry	Tank volume, tank surface, tank diagonal, obstacle area	Represents the size of the tank and obstacle.
Distance and event-strength proxies	Sensor distance, Event_Strength_Proxy, Blast_Scaled_Distance, inverse-distance terms	Represents sensor distance relative to a rough event-strength estimate.
Thermodynamics	Reduced pressure, superheat ratio, temperature ratio, vapour fraction	Represents the liquid/vapour state and critical conditions.
Obstacle interactions	Obstacle blockage, angle sine/cosine, blockage-angle terms	Models reflection and shielding effects caused by the wall.
Sensor/event position	Side one-hot features, Sensor ID one-hot features, sine/cosine encodings	Allows repeated sensor-location patterns to be learned.
Fluid class	Critical pressure and critical temperature key	Groups substances with similar physical properties.

Feature engineering created 154 numeric features from the raw physical columns. The goal was to give the models clearer signals about tank size, sensor location, obstacle effects, temperature conditions, and repeated sensor-position patterns.

Geometry features describe the size of the tank and obstacle, such as tank volume, tank surface area, obstacle area, and sensor distance. These features help the models understand the scale of each BLEVE setup.

The **scaled-distance features** compare sensor distance with a rough estimate of event strength. This is useful because the same sensor distance can mean different things for small and large BLEVE events. Since the dataset does not provide the actual explosive strength, I use Event_Strength_Proxy as an approximate substitute. This feature is used only to help the machine learning model, not as an exact physics calculation.

Thermodynamic features describe the liquid and vapour conditions. Temperature ratios compare the actual temperatures with boiling and critical temperatures, while reduced pressure compares tank pressure with critical pressure. These features help show whether the liquid is relatively cold, near boiling, or superheated.

Sensor encoding creates numeric indicators for sensor positions. Each sensor side and Sensor ID is represented so the models can learn repeated pressure patterns around the obstacle. I also used sine and cosine encodings for the repeated 27-position pattern.

Interaction features combine variables that may influence each other, such as liquid state with temperature or scaled distance with obstacle blockage.

Log transformations were applied to positive features with large ranges so the models could handle them more easily.

The final model uses all 154 numeric features in the neural-network branches, while the Ridge residual branch uses a smaller physics-focused feature set.

4.4. Encoding and Data Type Conversion

Model input columns were converted to numeric values so that scikit-learn models could use them consistently. Categorical and repeated-position information was encoded as numeric features. Status was converted into a binary value, sensor side and sensor ID were one-hot encoded, and sine/cosine encodings were used for Sensor ID and Event_Pos to represent the repeated 27-position pattern.

4.5. Scaling and Target Transformations

Ridge regression and neural networks are sensitive to feature scale, so I scaled the features before training these models. The main neural-network branches used RobustScaler because it is less affected by outliers. The smaller physics branch used StandardScaler before Ridge regression. Random Forest did not require scaling because tree models split using feature thresholds.

The target pressure is positive and strongly right skewed. The final model therefore used both Box-Cox target transformation and log-target modelling. These transformations make the target distribution easier for the neural networks to learn, and the predictions are then transformed back to pressure units.

5. Model Selection

The analysis compares three fundamentally different machine learning model types, followed by a blended ensemble. This allows the final model to be compared against simpler and more interpretable baselines.

5.1. Linear Regression Model

Ridge regression was the linear baseline. It is simple, stable, and regularised, so it is useful for checking how much of the pressure can be explained by broad linear relationships. I did not select it as the final model because BLEVE pressure depends on nonlinear interactions between geometry, temperature, obstacle position, and sensor location.

5.2. Random Forest Model

Random Forest was the tree-based baseline. It can capture nonlinear thresholds and feature interactions without requiring feature scaling. It performed much better than Ridge regression, but I did not select it as the final model because it was weaker than the neural network and final ensemble on validation and Kaggle testing.

5.3. Neural Network Model

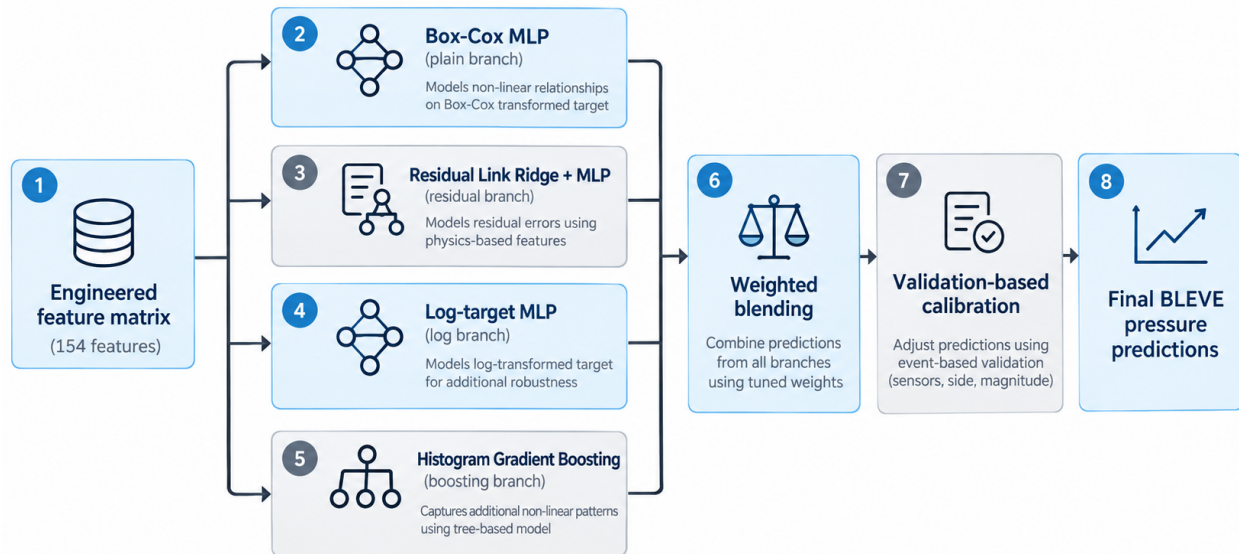
The neural network baseline used MLPRegressor with ReLU activation, L2 regularisation, early stopping, and a tuned hidden-layer structure. This suited the problem because engineered BLEVE features interact in nonlinear ways. It performed strongly, but a single neural network can still be sensitive to scaling, random initialisation, and the skewed target distribution.

5.4. Blended Ensemble Model

The final model used a blended ensemble because no single model family captured every part of the problem equally well. It combines several prediction branches: Box-Cox target MLPs, a residual branch based on Ridge plus MLP residual correction, a log-target MLP branch, and a Histogram Gradient Boosting branch. The branch predictions are blended and then adjusted using validation data.

Final Ensemble Architecture

Main branches used in the final prediction model



Multiple random seeds are averaged within the neural-network and boosting branches.

Figure 2- Final ensemble architecture combining neural-network, residual and gradient-boosting branches.

The residual branch first fits a simpler physics-focused Ridge model. Neural networks then learn the remaining error. In simple terms, this branch models a broad physical trend first, then learns the nonlinear leftover error. This helped the model learn both general blast behaviour and sensor-specific effects.

I selected the ensemble as the final model because it achieved the best local validation metrics and the best Kaggle public MAPE among the submitted model outputs.

6. Hyperparameter Tuning

Hyperparameter tuning required careful attention to the event-based structure of the data. Rather than randomly splitting rows, which would have allowed information leakage between training and validation sets, I held out complete BLEVE events for validation. This matters because sensors from the same event share the same physical conditions and setup, so any split that breaks up events would artificially inflate validation performance.

6.1. Validation Strategy

I created the validation set by selecting complete Event_ID groups and setting them aside entirely. The model trained on one set of events and was evaluated on completely different events it had never seen. While event-based k-fold cross-validation would have been more thorough, I used this simpler hold-out approach because it was significantly faster while still respecting the grouped data structure. For the project constraints, this was a reasonable trade-off.

Event-Based Validation Strategy

Why complete BLEVE events are kept together during validation

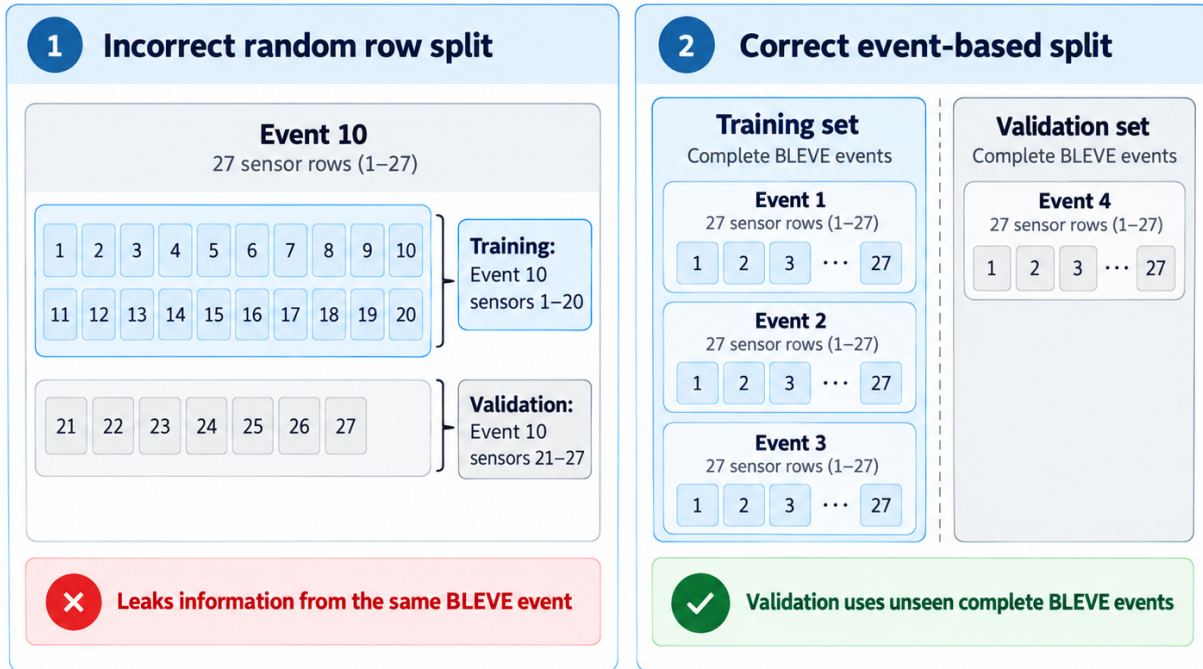


Figure 3- Event-based validation keeps complete BLEVE events together to reduce information leakage.

6.2. Search Strategy

The tuning approach was manual, and validation driven. I explored reasonable parameter ranges for each model, monitoring validation MAPE alongside supporting metrics like R2, RMSE, and MAE. Since Kaggle scores submissions on MAPE, I treated this as the primary metric. The other metrics served as sanity checks to make sure I was not overfitting or creating unstable predictions. I kept parameter settings that improved validation performance without adding unnecessary complexity, avoiding the trap of over-tuning on validation data.

6.3. Tuned Hyperparameters and Calibration

Model	Main hyperparameters tuned	Final selected settings
Ridge Regression	Regularisation strength alpha	Baseline Ridge alpha = 1.0; final residual RidgeCV searched alphas 0.1, 0.5, 1, 2, 5, and 10.
Random Forest	Number of trees, maximum depth, minimum samples split	n_estimators = 1000, max_depth = 30, min_samples_split = 5, random_state = 42.
Neural Network	Hidden layers, activation, alpha, learning rate, early stopping	Hidden layers = (256, 256, 128, 128, 128, 64, 32), ReLU, alpha = 0.0039, learning_rate_init = 0.00112, max_iter = 2000, early stopping enabled.
Final Ensemble	MLP architectures, random seeds, target transforms, branch weights, HGB settings, calibration strength	Seed averaging across Box-Cox MLP, log-target MLP, and HGB branches; HGB_W = 0.10; Box-Cox/log blend searched from 0.65 to 1.00; final calibration strength = 1.68.

Besides the model hyperparameters, the final ensemble also tuned the blend weight and applied a small calibration step using the validation set. The code automatically searched for the best blend between the Box-Cox and log-target predictions, testing values from 0.65 to 1.00. After blending the predictions, the calibration step grouped predictions into pressure ranges and learned a small adjustment for each range.

7. Model Prediction Performance

The local results below use the event-based hold-out validation split, so each model is compared on the same validation events. I also report the Kaggle public MAPE for each submitted model to compare how they performed on the competition test set.

7.1. Local Validation Performance

Model	MAPE	R2	RMSE	MAE
Linear Regression (Ridge)	0.742062	0.781328	0.216587	0.132332
Random Forest	0.239264	0.853797	0.177098	0.076528
Neural Network	0.174315	0.950580	0.102964	0.048740
Final Ensemble Model	0.126223	0.962758	0.089382	0.041311

The validation scores show a clear improvement from the simple linear baseline to the final ensemble. Ridge regression had the highest error because the problem is strongly nonlinear. Random Forest improved the result by modelling thresholds and interactions. The neural network improved further after scaling and tuning. The final ensemble achieved the lowest validation MAPE, RMSE, and MAE, and the highest R2.

7.2. Kaggle Public Test Performance

Model	Kaggle public MAPE
Ridge Linear Regression	0.70373
Random Forest	0.25362
Neural Network	0.17368
Final Ensemble Model	0.13285

Model Performance Comparison

Lower MAPE indicates better predictive accuracy

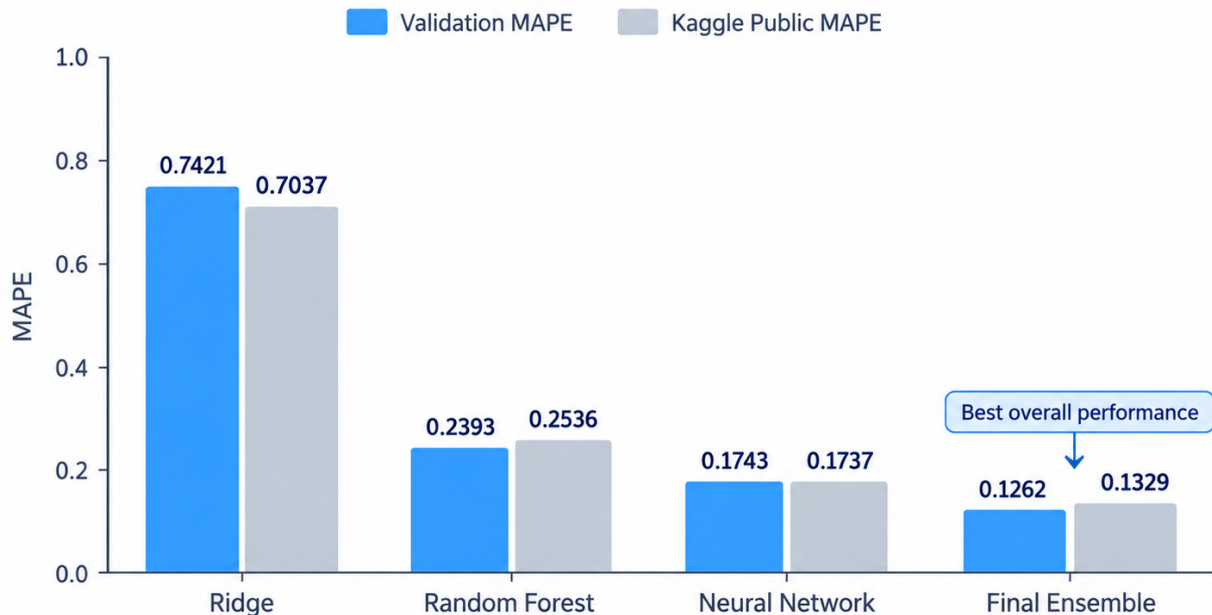


Figure 4- Comparison of validation and Kaggle public MAPE across the evaluated models.

The Kaggle public scores show a similar trend to the validation scores. The final ensemble achieved the lowest public MAPE, which suggests that the transformed-target modelling, residual learning, boosting branch, and calibration step helped improve generalisation.

8. Reflection

This project showed me that good model performance came from understanding the structure of the data, not only from choosing a powerful algorithm. Event grouping, unit checks, feature engineering, target transformation, and validation design all affected the reliability of the result.

8.1. Challenges

The main challenge was preventing leakage between related sensor measurements. Each BLEVE event produces multiple sensor readings, so random row splitting would mix sensors from the same event between training and validation. This would make validation scores look artificially strong. Hence, I used event-based validation so that complete events were kept together.

Another challenge was recognising and handling incomplete training events. Some events did not contain all 27 sensor readings, so applying a general completeness rule made the training data more consistent.

8.2. Learning

I learned that tabular modelling depends heavily on preprocessing choices. Correctly interpreting units, grouping rows by event, creating meaningful features, and choosing fair validation can be just as important as the final model architecture.

I also learned how transformed targets, residual correction, and prediction correction can work together. Box-Cox and log transforms made the skewed target easier to model, residual learning helped the neural network focus on leftover errors, and the final correction step reduced small systematic biases by sensor side and Sensor ID.

8.3. Improvements

One improvement would be to use full event-based k-fold cross-validation instead of one event-based hold-out split. The current approach avoids sensor leakage, but the validation score still depends on the single set of events chosen for validation. Event-based k-fold would repeat validation across several event splits and average the scores, giving a more reliable estimate of generalisation. The drawback is that it would require much more training time because the models would need to be refitted several times.

Another improvement would be to test additional model families such as XGBoost, LightGBM, or CatBoost. These gradient boosting methods are often strong for structured tabular regression because they can capture nonlinear interactions efficiently. I did not include them in the final workflow because the project already compared representative linear, tree-based, neural-network, and ensemble approaches. Adding more model families would require systematic tuning and fair event-based validation, so I left this as future work rather than adding them without proper comparison.

9. References

- Python. (2026). *difflib – Helpers for computing deltas*. https://docs.python.org/3/library/diffib.html#diffib.get_close_matches
- Pandas. (2026). *pandas.DataFrame.groupby*. <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.groupby.html>
- Scikit-Learn. (2026). *HistGradientBoostRegressor*. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html>
- Scikit-Learn. (2026). *MLPRegressor*. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html
- Scikit-Learn. (2026). *PowerTransformer*. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.PowerTransformer.html>
- Scikit-Learn. (2026). *RandomForestRegressor*. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>
- Scikit-Learn. (2026). *Ridge*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html
- Scikit-Learn. (2026). *RidgeCV*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.RidgeCV.html
- Scikit-Learn. (2026). *RobustScaler*. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.RobustScaler.html>
- Scikit-Learn. (2026). *StandardScaler*. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>
- Widanage, C., Mohotti, D., Lee, C. K., Wijesooriya, K., & Meddage, D. P. P. (2024). Use of explainable machine learning models in blast load prediction. *Engineering Structures*, 312, Article 118271. <https://doi.org/10.1016/j.engstruct.2024.118271>